Report on Network Traffic Analysis using Wireshark

Course: Computer Networks Lab

Name: Yagyesh Anshul

Roll No: 2302MC12

1. What I Did

This report walks through a hands-on analysis of my computer's network traffic using Wireshark. The goal was to peek under the hood of my internet connection to see how common protocols work in the real world. To do this, I captured live data while browsing a few websites and running a simple ping command.

2. How I Captured the Data

To see what was happening on the network, I captured about two minutes of traffic, which I saved in the file lab6.pcap. While the capture was running, I did two main things:

- **Browsed the web:** I visited a couple of websites to generate normal HTTP and HTTPS traffic.
- **Used Ping:** I ran ping 8.8.8.8 to send some ICMP packets out and see the replies.

After capturing the data, I used Wireshark's filters to zero in on the specific packets for HTTP, ICMP, and DNS so I could take a closer look at each one.

3. A Closer Look at the Packets

I picked out one packet from each of the main protocols I was looking for to see what they were made of. The table below breaks down the key details I found, like who was talking to whom (the IP addresses) and other interesting bits of information.

Table 1: Packet Detail Records

Protocol	Source IP	Destination IP	TTL	Packet Length	What it was doing (Flags/Details)
HTTP	192.170.9.15	34.223.124.45	128	507 bytes	Pushing data and acknowledging it (PSH, ACK)

ICMP	192.170.9.15	8.8.8.8	128	74 bytes	Sending a ping (Echo request)
DNS	8.8.8.8	192.170.9.15	106	192 bytes	Replying to a name lookup (Standard query response)

4. What I Found

The Busiest Protocols

As I expected from browsing the web, the most common protocols were all related to loading websites. If I were to look at the protocol hierarchy in Wireshark, the top three would undoubtedly be:

1. **TCP (Transmission Control Protocol):** This was the clear winner. It's the workhorse protocol that makes sure all the data for websites gets delivered reliably.
2. **TLS (Transport Layer Security):** Since almost every site uses HTTPS now, TLS traffic was everywhere. It's the protocol that encrypts my HTTP requests to keep my browsing private.
3. **UDP (User Datagram Protocol):** This showed up mainly for DNS lookups. It's fast and perfect for the quick back-and-forth of asking for a website's IP address.

Anything Suspicious?

I took a good look through the captured traffic, and everything seemed normal. There were no strange IP addresses or unexpected protocols running. All the traffic I saw was directly connected to what I was doing, whether it was my computer asking a DNS server for an IP address or my browser talking to a web server.

5. Key Takeaways

This whole exercise was really insightful. Here are a few things that stood out to me:

- **Web Browsing is Deceptively Complex:** It was interesting to see that just clicking a link sets off a whole chain reaction. Your computer first has to send out a DNS query to find the server, then it builds a connection with TCP, and only then does it actually start fetching the website's content. A lot happens in the background that we never see.
- **The Network Layers Aren't Just Theory:** Wireshark really brought the textbook concept of the TCP/IP model to life. I could actually see the different layers in each packet from the Ethernet frame at the bottom all the way up to the application data at the top. It was cool to see how the data gets wrapped up at each step.

- **Different Tools for Different Jobs:** It was also very clear why there are different protocols. TCP is built to be slow but steady, making sure nothing gets lost, which is perfect for a webpage. UDP is all about speed for things like DNS. And ICMP is in its own category, just for checking if another machine is online.